

AWS Cloud Handbook

1 AWS Handbook — Master Table of Contents

- [1.1 Part I — Cloud Foundations \(Pages 1–80\)](#)
- [1.2 Part II — Identity & Access Management \(Pages 81–160\)](#)
- [1.3 Part III — Networking \(Pages 161–280\)](#)
- [1.4 Part IV — Compute \(Pages 281–400\)](#)
- [1.5 Part V — Storage \(Pages 401–480\)](#)
- [1.6 Part VI — Databases \(Pages 481–560\)](#)
- [1.7 Part VII — Application Integration \(Pages 561–620\)](#)
- [1.8 Part VIII — DevOps on AWS \(Pages 621–700\)](#)
- [1.9 Part IX — Observability & Operations \(Pages 701–760\)](#)
- [1.10 Part X — Security & Compliance \(Pages 761–800\)](#)
- [1.11 Expansion status](#)

2 Chapter 1: Introduction to AWS & the Shared Responsibility Model

- [2.1 1.1 What is Amazon Web Services \(AWS\)?](#)
 - [2.1.1 Why AWS matters for DevOps](#)
- [2.2 1.2 Cloud service models](#)
 - [2.3 1.3 The Shared Responsibility Model](#)
 - [2.3.1 AWS is responsible for **Security OF the Cloud**](#)
 - [2.3.2 You are responsible for **Security IN the Cloud**](#)
 - [2.3.3 Example: RDS vs EC2 MySQL](#)
- [2.4 1.4 Core AWS concepts](#)
- [2.5 1.5 AWS console, CLI, and IaC](#)
- [2.6 1.6 Pricing fundamentals](#)
- [2.7 1.7 Well-Architected Framework \(preview\)](#)
- [2.8 1.8 Chapter summary](#)
- [2.9 Lab 1.1 — Create your AWS account](#)
- [2.10 Review questions](#)

3 Chapter 2: AWS Global Infrastructure — Regions, AZs, and Edge

- [3.1 2.1 Global infrastructure overview](#)
- [3.2 2.2 Regions](#)
 - [3.2.1 Choosing a region](#)
 - [3.2.2 Regional vs global services](#)
- [3.3 2.3 Availability Zones \(AZs\)](#)
 - [3.3.1 Multi-AZ patterns](#)
- [3.4 2.4 Local Zones, Wavelength, and Outposts](#)
- [3.5 2.5 Edge locations and CloudFront](#)
- [3.6 2.6 Fault isolation design principles](#)
 - [3.6.1 S3 durability vs availability](#)
- [3.7 2.7 Service quotas \(limits\)](#)
- [3.8 2.8 Chapter summary](#)
- [3.9 Lab 2.1 — Explore regions in CLI](#)
- [3.10 Review questions](#)

4 Chapter 5: IAM Users, Groups, Roles, and Policies

- [4.1 5.1 What is IAM?](#)
- [4.2 5.2 IAM identity types](#)
- [4.3 5.3 Policies — the core of permissions](#)
 - [4.3.1 Policy structure](#)
 - [4.3.2 Policy types](#)
- [4.4 5.4 Users and groups](#)
 - [4.4.1 Create a developer group \(example workflow\)](#)
- [4.5 5.5 Roles and trust policies](#)
 - [4.5.1 EC2 instance role example](#)
 - [4.5.2 Assume role \(cross-account or CLI\)](#)

[4.6 5.6 AWS managed vs customer managed policies](#)

[4.7 5.7 Least privilege in practice](#)

[4.7.1 Tag-based access \(ABAC\)](#)

[4.8 5.8 Common IAM actions for DevOps](#)

[4.9 5.9 IAM Access Analyzer & CloudTrail](#)

[4.10 5.10 Chapter summary](#)

[4.11 Lab 5.1 — IAM hands-on](#)

[4.12 Review questions](#)

1 AWS Handbook — Master Table of Contents

Target length: ~800 pages (~320,000 words)
Audience: Beginner → Advanced AWS practitioners
Companion: [DevOps Handbook](#)

1.1 Part I — Cloud Foundations (Pages 1-80)

Ch	Title	Pages
1	Introduction to AWS & Shared Responsibility	1-18
2	AWS Global Infrastructure: Regions, AZs, Edge	19-35
3	AWS Account Setup, Billing, and Cost Basics	36-55
4	AWS CLI, CloudShell, and SDK Overview	56-80

Written: [Chapter 1](#), [Chapter 2](#)

1.2 Part II — Identity & Access Management (Pages 81-160)

Ch	Title	Pages
5	IAM Users, Groups, Roles, and Policies	81-105
6	IAM Best Practices & Permission Boundaries	106-125
7	IAM Identity Center (SSO) & Federation	126-145
8	AWS Organizations & Service Control Policies	146-160

Written: [Chapter 5](#)

1.3 Part III — Networking (Pages 161-280)

Ch	Title	Pages
9	VPC Fundamentals: CIDR, Subnets, Route Tables	161-185
10	Internet Gateway, NAT, and Routing	186-205
11	Security Groups vs NACLs	206-220
12	Load Balancers: ALB, NLB, GWLB	221-245
13	Route 53: DNS Records and Routing Policies	246-265
14	VPC Peering, Transit Gateway, PrivateLink	266-280

1.4 Part IV — Compute (Pages 281-400)

Ch	Title	Pages
----	-------	-------

15	EC2: Instance Types, AMIs, Launch Templates	281–305
16	Auto Scaling & Elastic Load Balancing Integration	306–325
17	Lambda: Events, Concurrency, and Layers	326–350
18	ECS & Fargate	351–375
19	EKS: Managed Kubernetes on AWS	376–400

1.5 Part V — Storage (Pages 401–480)

Ch	Title	Pages
20	S3: Buckets, Objects, Versioning, Lifecycle	401–425
21	S3 Security, Encryption, and Static Websites	426–445
22	EBS & EFS	446–465
23	AWS Backup & Disaster Recovery Patterns	466–480

1.6 Part VI — Databases (Pages 481–560)

Ch	Title	Pages
24	RDS & Aurora	481–505
25	DynamoDB: Tables, Indexes, Streams	506–525
26	ElastiCache & Caching Patterns	526–540
27	Database Migration (DMS) & DR	541–560

1.7 Part VII — Application Integration (Pages 561–620)

Ch	Title	Pages
28	API Gateway	561–580
29	SQS, SNS, and EventBridge	581–605
30	Step Functions Workflows	606–620

1.8 Part VIII — DevOps on AWS (Pages 621–700)

Ch	Title	Pages
31	CloudFormation Deep Dive	621–645
32	AWS CDK	646–665
33	CodePipeline, CodeBuild, CodeDeploy	666–685
34	Systems Manager & Parameter Store	686–700

1.9 Part IX — Observability & Operations (Pages 701-760)

Ch	Title	Pages
35	CloudWatch Metrics, Alarms, Dashboards	701-720
36	CloudWatch Logs & Insights	721-735
37	X-Ray & Distributed Tracing	736-750
38	CloudTrail & AWS Config	751-760

1.10 Part X — Security & Compliance (Pages 761-800)

Ch	Title	Pages
39	KMS, Secrets Manager, ACM	761-775
40	GuardDuty, Security Hub, WAF, Shield	776-790
41	Well-Architected Review & Capstone Design	791-800

1.11 Expansion status

Part	Status
I	Ch 1-2 written (~35 pages)
II	Ch 5 written (~25 pages)
III-X	Outlined; request by chapter number

To request a chapter: *“Expand AWS Handbook Chapter 15 (EC2 deep dive)”*

2 Chapter 1: Introduction to AWS & the Shared Responsibility Model

AWS Handbook — Part I, Pages 1–18

2.1 1.1 What is Amazon Web Services (AWS)?

Amazon Web Services (AWS) is a comprehensive **cloud computing platform** offering on-demand compute, storage, databases, networking, machine learning, and 200+ other services. Instead of buying physical servers, you provision resources in minutes and pay primarily for what you use.

2.1.1 Why AWS matters for DevOps

- **API-driven** — Every service is controllable via CLI, SDK, and IaC.
 - **Global scale** — Deploy in multiple regions for resilience and latency.
 - **Managed services** — Offload undifferentiated heavy lifting (RDS, EKS, Lambda).
 - **Ecosystem** — Largest cloud market share; extensive documentation and community.
-

2.2 1.2 Cloud service models

Model	You manage	Provider manages	AWS examples
IaaS	OS, apps, data	Hardware, network, hypervisor	EC2, VPC, EBS
PaaS	Apps, data	Runtime, OS, infra	Elastic Beanstalk, App Runner
SaaS	Configuration, data	Everything else	WorkSpaces, Chime
FaaS	Function code	Everything else	Lambda

Containers blur lines: ECS/EKS give you orchestration (between IaaS and PaaS).

2.3 1.3 The Shared Responsibility Model

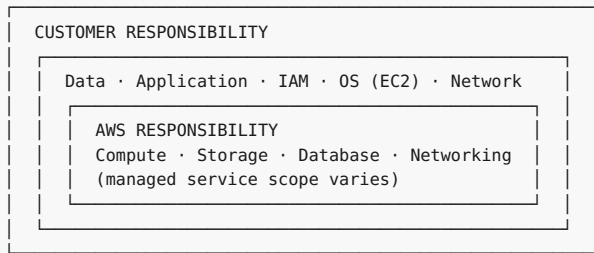
Security and compliance are **shared** between AWS and the customer. The boundary shifts by service type.

2.3.1 AWS is responsible for Security OF the Cloud

- Physical data centers
- Hardware and global network
- Virtualization layer
- Managed service patching (for fully managed services)

2.3.2 You are responsible for Security IN the Cloud

- Data classification and encryption choices
- IAM users, roles, and policies
- Operating system patches (on EC2)
- Network configuration (security groups, NACLs)
- Application code security



2.3.3 Example: RDS vs EC2 MySQL

Task	EC2 + self-managed MySQL	Amazon RDS
Patch MySQL	You	AWS
Patch OS	You	AWS
Configure security groups	You	You
Encrypt data at rest	You (configure)	You (enable)
Physical security	AWS	AWS

2.4 1.4 Core AWS concepts

Concept	Definition
Region	Geographic area (e.g., <code>us-east-1</code> , <code>eu-west-1</code>)
Availability Zone (AZ)	Isolated data center within a region
Account	Billing and security boundary (12-digit ID)
Resource	Any AWS entity (EC2 instance, S3 bucket, etc.)
ARN	Amazon Resource Name — unique identifier
Tag	Key-value metadata on resources

Example ARN:

```
arn:aws:s3:::my-bucket
arn:aws:ec2:us-east-1:123456789012:instance/i-0abcd1234
```

2.5 1.5 AWS console, CLI, and IaC

Three ways to interact with AWS:

1. **Management Console** — Web UI; great for learning and ad-hoc tasks.
2. **AWS CLI** — Scriptable; essential for automation.
3. **Infrastructure as Code** — CloudFormation, CDK, Terraform for reproducible environments.

DevOps engineers spend most time in **CLI + IaC**, using the console for debugging.

2.6 1.6 Pricing fundamentals

- **On-Demand** — Pay per hour/second; no commitment.
- **Reserved Instances / Savings Plans** — 1–3 year commitment; lower cost.
- **Spot Instances** — Unused capacity; cheap but can be interrupted.
- **Free Tier** — Limited free usage for 12 months (and some always-free services).

Always set billing alarms (AWS Budgets) on new accounts.

2.7 1.7 Well-Architected Framework (preview)

AWS defines six pillars for good architecture (covered throughout this handbook):

1. Operational Excellence
 2. Security
 3. Reliability
 4. Performance Efficiency
 5. Cost Optimization
 6. Sustainability
-

2.8 1.8 Chapter summary

- AWS provides on-demand cloud infrastructure and managed services.
 - **Shared responsibility** splits security between AWS and you—know your side.
 - Learn **regions, accounts, ARNs, and tags** early.
 - Use CLI and IaC for production; console for exploration.
-

2.9 Lab 1.1 — Create your AWS account

1. Sign up at <https://aws.amazon.com> (use a dedicated email).
 2. Enable **MFA** on the root account; do not use root daily.
 3. Create an **IAM admin user** for daily work.
 4. Set a **billing budget** alert at \$10 or your comfort threshold.
 5. Explore the console: EC2, S3, IAM dashboards.
-

2.10 Review questions

1. What is the difference between IaaS and PaaS? Give one AWS example of each.
 2. Who patches the OS on an EC2 instance?
 3. What is an ARN used for?
 4. Name two AWS pricing models.
-

Next: [Chapter 2 — Global Infrastructure](#)

3 Chapter 2: AWS Global Infrastructure — Regions, AZs, and Edge

AWS Handbook — Part I, Pages 19–35

3.1 2.1 Global infrastructure overview

AWS operates a worldwide network of data centers organized into:

Region (e.g., us-east-1)

- Availability Zone A (us-east-1a)
- Availability Zone B (us-east-1b)
- Availability Zone C (us-east-1c)
- ... (typically 3+ AZs per region)

Edge Locations (200+) — CloudFront, Route 53, etc.

3.2 2.2 Regions

A **Region** is a separate geographic area. Each region is **fully isolated** from others for most services.

3.2.1 Choosing a region

Factor	Guidance
Latency	Place resources near users
Compliance	Data residency (GDPR, etc.) may require EU regions
Service availability	New services launch in select regions first
Cost	Pricing varies by region (us-east-1 often cheapest)
Disaster recovery	Use a second region for critical workloads

Naming: us-east-1 (N. Virginia), eu-west-1 (Ireland), ap-southeast-1 (Singapore).

3.2.2 Regional vs global services

Regional (must specify region)	Global (single endpoint)
EC2, RDS, VPC, Lambda	IAM, Route 53, CloudFront
S3 (bucket tied to region)	AWS Organizations

3.3 2.3 Availability Zones (AZs)

An **AZ** is one or more discrete data centers with independent power, networking, and cooling within a region. AZs are connected by **low-latency private fiber**.

- Deploy across **multiple AZs** for high availability.
- AZ names like us-east-1a are **account-specific mappings**— 1a in your account may not be the same physical DC as 1a in another account.

3.3.1 Multi-AZ patterns

Pattern	Use case
Active-passive	RDS Multi-AZ failover
Active-active	ALB spanning AZs with EC2 in each
Stretch cluster	EKS node groups per AZ

3.4 2.4 Local Zones, Wavelength, and Outposts

Type	Purpose
Local Zones	Extend region closer to metro areas (ultra-low latency)
Wavelength	5G edge at carrier locations
Outposts	AWS hardware on-premises; hybrid cloud

These are specialized; start with standard regions + AZs.

3.5 2.5 Edge locations and CloudFront

Edge locations cache content close to users via **Amazon CloudFront** (CDN). Route 53 and AWS Shield also use the edge network.

Benefits: - Lower latency for static assets - Reduced origin load - DDoS protection at edge (with Shield)

3.6 2.6 Fault isolation design principles

1. **Never rely on a single AZ** for production.
2. **Design for regional failure** for mission-critical systems (multi-region active-active or pilot light).
3. **Use managed services** that handle AZ redundancy (RDS Multi-AZ, S3 cross-AZ durability).
4. **Test failover** regularly.

3.6.1 S3 durability vs availability

- **S3 Standard** — 99.999999999% (11 nines) durability across AZs in a region.
- **Availability** — Different metric; rare regional outages can still occur.

3.7 2.7 Service quotas (limits)

Each account has **default quotas** per region (e.g., EC2 vCPUs, VPCs per region). Request increases via **Service Quotas** console. Plan capacity before large deployments.

3.8 2.8 Chapter summary

- **Regions** are geographic; **AZs** are isolated DCs within a region.

- Deploy production workloads across **multiple AZs**.
 - **Edge locations** power CloudFront and global services.
 - Know **regional vs global** services when designing architecture.
-

3.9 Lab 2.1 — Explore regions in CLI

```
aws configure                                # Set default region
aws ec2 describe-availability-zones --region us-east-1
aws ec2 describe-regions --output table
```

Document which region you will use for labs and why.

3.10 Review questions

1. How many AZs should a highly available web app use at minimum?
 2. Is IAM a regional or global service?
 3. What AWS service uses edge locations for caching?
 4. Why might you choose `eu-central-1` over `us-east-1`?
-

Next: Part II — [Chapter 5: IAM Fundamentals](#)

4 Chapter 5: IAM Users, Groups, Roles, and Policies

AWS Handbook — Part II, Pages 81–105

4.1 5.1 What is IAM?

AWS Identity and Access Management (IAM) controls **who** can access **which** AWS resources and **how**. Every API call to AWS is authenticated and authorized through IAM (except public resources like anonymous S3 access).

IAM is **global**—users and roles exist at the account level, not per region.

4.2 5.2 IAM identity types

Identity	Use case
Root user	Account owner; created at signup. Avoid for daily use.
IAM user	Long-term credentials for humans or legacy apps
IAM role	Temporary credentials; assumed by users, services, or other accounts
IAM group	Collection of users with shared permissions

Best practice: Humans use **IAM Identity Center (SSO)** or roles with MFA. Applications and EC2/Lambda use **roles**, not access keys on users.

4.3 5.3 Policies — the core of permissions

A **policy** is a JSON document defining **Allow** or **Deny** for **actions** on **resources** under **conditions**.

4.3.1 Policy structure

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowS3Read",
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3::my-app-bucket",
        "arn:aws:s3::my-app-bucket/*"
      ],
      "Condition": {
        "IpAddress": {
          "aws:SourceIp": "203.0.113.0/24"
        }
      }
    }
  ]
}
```

```
}
```

4.3.2 Policy types

Type	Attached to
Identity-based	Users, groups, roles
Resource-based	S3 buckets, SNS topics, etc.
Permission boundaries	Max permissions for identity
SCP	AWS Organizations — account ceiling
Session policies	Limit role session scope

Evaluation logic: Explicit **Deny** always wins. Default is implicit deny until Allow.

4.4 5.4 Users and groups

4.4.1 Create a developer group (example workflow)

1. Create group `Developers`.
2. Attach policy `PowerUserAccess` (or custom least-privilege policy).
3. Create users; add to group.
4. Enable **MFA** for console access.
5. Provide **programmatic access** only if needed (prefer roles).

```
aws iam create-group --group-name Developers
aws iam attach-group-policy --group-name Developers \
  --policy-arn arn:aws:iam::aws:policy/PowerUserAccess
aws iam create-user --user-name alice
aws iam add-user-to-group --user-name alice --group-name Developers
```

4.5 5.5 Roles and trust policies

A **role** has two policies:

1. **Trust policy** — Who can assume the role
2. **Permission policy** — What the role can do

4.5.1 EC2 instance role example

Trust policy (allows EC2 service to assume):

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": { "Service": "ec2.amazonaws.com" },
      "Action": "sts:AssumeRole"
    }
  ]
}
```

Attach permission policy allowing `s3:GetObject` on a specific bucket. The EC2 instance receives **temporary credentials** via instance metadata—no hardcoded keys.

4.5.2 Assume role (cross-account or CLI)

```
aws sts assume-role \  
  --role-arn arn:aws:iam::123456789012:role/DeployRole \  
  --role-session-name deploy-session
```

4.6 5.6 AWS managed vs customer managed policies

Type	Description
AWS managed	Maintained by AWS (<code>ReadOnlyAccess</code> , <code>AdministratorAccess</code>)
Customer managed	You create and reuse across identities
Inline	Embedded in single user/group/role; harder to reuse

Prefer **customer managed** policies for team standards; avoid `AdministratorAccess` except break-glass accounts.

4.7 5.7 Least privilege in practice

1. Start with **read-only** access; add permissions as needed.
2. Use **IAM Access Analyzer** to find overly broad access.
3. Scope resources with ARNs, not `"Resource": "*" .`
4. Use **conditions** (MFA present, source IP, tag-based).
5. Rotate or eliminate **access keys**; prefer roles.

4.7.1 Tag-based access (ABAC)

```
"Condition": {  
  "StringEquals": {  
    "aws:PrincipalTag/Department": "${aws:ResourceTag/Department}"  
  }  
}
```

Users tagged `Department=Engineering` access only resources tagged the same.

4.8 5.8 Common IAM actions for DevOps

Action prefix	Service
<code>ec2:*</code>	EC2
<code>s3:*</code>	S3
<code>iam:PassRole</code>	Allow attaching roles to services
<code>sts:AssumeRole</code>	Switch roles
<code>cloudformation:*</code>	Stack operations

`iam:PassRole` is often required when CI/CD deploys resources that need an instance/task role.

4.9 5.9 IAM Access Analyzer & CloudTrail

- **Access Analyzer** — Identifies resources shared externally.
- **CloudTrail** — Logs every IAM API call for audit.

Enable CloudTrail in all regions for security accounts.

4.10 5.10 Chapter summary

- IAM controls authentication and authorization globally.
 - Use **groups** for humans, **roles** for services and temporary access.
 - Policies are JSON with Effect, Action, Resource, Condition.
 - Apply **least privilege**; avoid root and long-lived access keys.
-

4.11 Lab 5.1 — IAM hands-on

1. Create IAM user `lab-user` with MFA.
 2. Create customer managed policy allowing `s3:ListBucket` and `s3:GetObject` on one bucket.
 3. Create role `EC2-S3-ReadRole` with trust for EC2 and the S3 read policy.
 4. Launch EC2 with the role; verify `aws s3 ls` works without configured keys.
 5. Review effective permissions in **IAM Policy Simulator**.
-

4.12 Review questions

1. What is the difference between an IAM user and an IAM role?
 2. What happens when Allow and Deny conflict?
 3. Why should EC2 instances use roles instead of access keys?
 4. What does `sts:AssumeRole` do?
-

Continue: Chapter 6 — IAM Best Practices (outline in TOC)